

Lab exercises

Track 16.2 –  TypeScript for AI Applications · 16-ai-tech-stack/tracks/typescript/lab/EXERCISES.md

Lab — TypeScript for AI Applications

Work top to bottom. Every exercise has a **stated deliverable**; if you cannot produce it, you have not finished. No solutions — the checks tell you when you are right.

Everything runs offline. `cd ../code && npm install` first.

1. Warm-up: read the code you were given (45 min)

Run both scripts. Then:

- **1a.** In `typed-tool-calling.ts`, change `toWireTool` to use `io: "output"` instead of `io: "input"` and re-run. Which field moves in or out of `required`, and what would the model do wrong if you shipped that schema?
- **1b.** In `streaming-parser.ts`, delete the `{ stream: true }` option from the correct parser (leave everything else). Re-run section 4. Explain the output in one sentence.
- **1c.** Change `chunkBytes` in section 3 from `4096` to `220` and re-run. At what chunk size does the broken parser stop producing correct output, and why is that number not a constant you could rely on?

Deliverable: three sentences, one per part.

2. Break the exhaustiveness check (30 min)

Add a fourth tool — `cancel_deploy`, taking `{ service: string; reason: string }` — to `typed-tool-calling.ts`. Add it to `TOOLS` and to the `ToolCall` union, but **do not** add a `case` to `dispatch`.

Checks: - `npm run typecheck` fails with an error pointing at `assertNever(call)`. - The error message names `cancel_deploy`. - `npm run tsx typed-tool-calling.ts` **still runs** — explain in one sentence why the runtime is happy while the type checker is not, and what that tells you about putting `tsc --noEmit` in CI.

3. Fix a real parser bug (1.5h)

`sseParser` handles frames separated by `\n\n`. Real servers also send `\r\n\r\n` (and some proxies rewrite one into the other), plus SSE comment lines starting with `:` used as keep-alive heartbeats.

- **3a.** Extend the parser to accept `\r\n\r\n` as a frame separator.
- **3b.** Ignore comment lines (`:ping`) without breaking framing.
- **3c.** Add an `id: / retry:` field parse and expose the last seen event id.

Checks: - A stream mixing `\n\n` and `\r\n\r\n` separators parses identically at `chunkBytes` of 1, 7 and 4096.
- A heartbeat comment between two data frames does not produce a spurious event. - Test at `chunkBytes: 1`.
If it passes at 1, it passes anywhere.

4. Zod against a hostile model (1.5h)

Write `repair-loop.ts`. Simulate a model that returns tool arguments and fails **three** times before succeeding:

1. an enum value outside the allowed set,
2. a nested object missing a required field,
3. valid JSON of the wrong top-level shape (an array where an object was expected).

On each failure, feed `z.prettifyError(result.error)` back as a `tool_result` with `is_error: true`, and have your fake model "read" it to produce the next attempt.

Checks: - The loop terminates on success **and** on a `maxAttempts` cap — with a distinct outcome for each. - Nothing throws. The whole thing returns a `Result<T, E>`. - Add a fourth failure the loop *cannot* repair (`z.string().uuid()` against a model that keeps emitting slugs). Confirm you hit the cap cleanly.

Deliverable: the script, plus one paragraph on how many repair attempts is the right cap and what it costs you when you set it too high.

5. The route handler (2h)

Build a real `POST /api/chat` in Next.js, Hono, or plain `node:http`. It must:

- accept `{ messages: Message[] }`, validated with Zod at the boundary;
- return a `ReadableStream` with the correct SSE headers (including `no-transform` and `X-Accel-Buffering: no`);
- forward `req.signal` into the model call;
- emit your own `{"type":"error"}` frame if the upstream stream fails mid-body.

Use a simulated model (reuse `fakeSocket`) so this stays offline, or wire up `@anthropic-ai/sdk` if you have a key.

Checks: - `curl -N localhost:3000/api/chat -d '{"messages":[...]}'` shows tokens arriving progressively, not all at once. If they arrive at once, find the buffer. - Killing the curl process produces a "cancelled" log line, not an unhandled rejection. - Posting `{"messages":"nope"}` returns a 400 with the Zod error, not a 500.

6. Strictness archaeology (1h)

Take any existing TypeScript project you have — ideally one that already compiles. Turn on, one at a time:

```
"noUncheckedIndexedAccess": true,  
"exactOptionalPropertyTypes": true,  
"verbatimModuleSyntax": true
```

Deliverable: a table — flag | new errors | how many were real bugs | how many were noise. Then answer: which of the three would you turn on in a codebase you inherited tomorrow, and which would you defer?

7. Edge audit (1h)

Take your route handler from #5 and make it run on Cloudflare Workers (`wrangler dev`) or Vercel Edge.

Checks: - List every import you had to remove or replace. For each, say what the edge substitute was. - Measure cold start and time-to-first-token on both Node and edge. Report both numbers. - Answer: which parts of your app should **not** move to the edge, and why?

8. Capstone: the streaming agent (3h)

Combine everything into one small app:

1. A Zod tool registry with at least three tools, one of which fails at runtime.
2. A streaming loop: model → tool calls → tool results → model, streaming text through to the client the whole way.
3. Tool-call validation with repair, capped at 2 attempts.
4. A Stop button that aborts end to end.
5. `tsc --noEmit` clean under the strict config from `code/tsconfig.json`.
6. One vitest suite that replays a recorded SSE fixture at chunk sizes `[1, 3, 7, 64, 4096]` and asserts identical output for all five.

Check: hand it to someone else. If they can (a) make the model call a tool with bad arguments and watch it recover, and (b) hit Stop mid-generation and see your server log "cancelled", you passed.

9. Stretch: type-level tool names (1h)

Make `parseToolCall` in `typed-tool-calling.ts` return a properly narrowed `ToolCall` **without** the `as ToolCall` cast — using a mapped type over the `TTOOLS` tuple keyed by `name`.

Check: the cast is gone, `tsc` is clean, and adding a tool to `TTOOLS` still updates `ToolName` automatically.

Then answer honestly: was the type gymnastics worth deleting one audited cast in one function? Write down your answer. It is the same question you will face every time someone proposes a clever type in review.